

161. Kada ne treba menjati veličinu skupa fizičkih stranica procesa?

U slučaju da je učestanost straničnih prekida između pomenute dve granice, tada nema potrebe za izmenom broja fizičkih stranica u skupu fizičkih stranica aktivnog procesa. U ovom slučaju skup fizičkih stranica (odnosno, njima odgovarajući skup virtuelnih stranica) obrazuje radni skup (working set).

162. Koji pristupi oslobađanja fizičkih stranica obezbeđuju smanjenje učestanosti straničnih prekida nakon povećanja broja fizičkih stranica procesa?

- Oslobađanje fizičkih stranica može biti zasnovano samo na upotrebi bita referenciranja i bita izmene. U ovom pristupu se pronalazi fizička stranica koja nije korišćena u nekom prethodnom periodu (Not Recently Used – NRU).
- pronalaženja najmanje korišćene fizičke stranice (Least Recently Used-LRU)
- softverska simulacija pronalaženja najmanje korišćene fizičke stranice (Not Frequently Used - NFU/aging)

163. Koji pristupi oslobađanja fizičkih stranica koriste bit referenciranja?

Sledeći pristupi oslobađanja fizičkih stranica koriste bit referenciranja:

- pronalaženje fizičke stranice koja nije korišćena u nekom prethodnom periodu (Not RecentlyUsed - NRU),
- pronalaženja najmanje korišćene fizičke stranice (Least Recently Used-LRU),
- softverska simulacija pronalaženja najmanje korišćene fizičke stranice (Not Frequently Used NFU/aging),
- second chance,
- clock,
- wclock

164. Koji pristupi oslobađanja fizičkih stranica koriste bit izmene?

Sledeći pristupi oslobađanja fizičkih stranica koriste bit izmjene:

- Not Recently Used – NRU
- oslobađanje fizičke stranice sa najstarijim sadržajem (First In First Out - FIFO) (koristi i bit izmene),
- second chance,
- clock,
- wclock

165. Šta karakteriše tipične ciljeve raspoređivanja?

O izboru spremnog procesa, na koga se preključuje procesor, brine raspoređivanje (scheduling). Ovaj izbor zavisi od cilja raspoređivanja. Tipični ciljevi raspoređivanja su, na primer, (1) poboljšanje iskorišćenja procesorskog vremena, (2) ravnomerna raspodela procesorskog vremena, (3) što kraći odziv na korisničku akciju ili neki drugi oblik postizanja potrebnog kvaliteta usluge. Ovakvi ciljevi nisu saglasni, pa se ne mogu istovremeno ostvariti.

166. Šta je cilj raspoređivanja za neinteraktivno korišćenje računara?

Za neinteraktivno korišćenje računara cilj raspoređivanja je poboljšanje iskorišćenja procesorskog vremena. Ovakav cilj se ostvaruje minimiziranjem preključivanja na neophodan broj (samo nakon pozivanja blokirajućih sistemskih operacija ili nakon kraja aktivnosti procesa).

167. Šta je cilj raspoređivanja za interaktivno korišćenje računara?

Za interaktivno korišćenje računara (u višekorisničkom režimu rada) ciljevi raspoređivanja su ravnomerna raspodela procesorskog vremena između istovremeno postojećih procesa, odnosno, između njihovih vlasnika (korisnika, koji istovremeno koriste računar) i što kraći odziv na korisničku akciju. Ovakvi ciljevi se ostvaruju kružnim raspoređivanjem.

168. Zašto je uvedeno kružno raspoređivanje?

Kružno raspoređivanje je uvedeno radi ravnomjernog raspoređivanja procesorskog vremena između istovremeno postojećih procesa i što kraćeg odziva na korisničku akciju. Tada ono svakom od istovremeno postojećih procesa dodjeljuje isti vremenski interval, nazvan kvantum. Kružno raspoređivanje se uspješno primjenjuje i u situaciji kada hitnost svih procesa nije ista, pa se zbog toga procesima dodjeljuju razni prioriteti.

169. Šta doprinosi ravnomernoj raspodeli procesorskog vremena?

Ravnomijernoj raspodjeli procesorskog vremena doprinosi uvođenje kvantuma. Po isticanju kvantuma, aktivni proces prepušta procesor spremnom procesu, koji najduže čeka na svoj kvantum. Dinamička izmjena prioriteta procesa doprinosi ravnomijernosti raspodjele procesorskog vremena:

- između procesa, ako se uspostavi obrnuta proporcionalnost između prioriteta procesa i obima u kome je on iskoristio poslednji kvantum.
- između korisnika, ako se uspostavi obrnuta proporcionalnost između prioriteta procesa, koji pripadaju nekom korisniku, i ukupnog dijela u procesorskom vremenu tog korisnika u toku njegove interakcije sa računarom.

Ravnomijerna raspodjela procesorskog vremena može se postići i bez izmjena prioriteta, lutrijskim raspoređivanjem.

170. Šta je cilj raspoređivanja za multimedijalne aplikacije?

Za multimedijalne aplikacije, koje zahtevaju visoku propusnost podataka i njihovu isporuku sa pravilnim periodom, cilj raspoređivanja je garantovanje procesima potrebnog broja kvantuma u pravilnim vremenskim razmacima.

171. Do čega dovodi skraćenje kvantuma?

Skraćenje kvantuma doprinosi poboljšanju odziva, ali i smanjenju iskorišćenja procesora, jer povećava broj preključivanja koja troše procesorsko vrijeme. Suviše kratak kvantum počinje da ugrožava i odziv, kada se prevelik procenat procesorskog vremena počne trošiti na preključivanje.

172. Šta se postiže uticanjem na nivo prioriteta i na dužinu kvantuma?

Uticanjem na nivo prioriteta i na dužinu kvantuma obezbjeđujemo dobar odziv procesora na korisničke procese, kao i kvalitetno iskorišćenje procesorskog vremena za pozadinske procese. Što je proces duže aktivan duži mu je kvantum i niži prioritet

173. Šta karakteriše operativne sisteme realnog vremena?

Operativni sistemi realnog vremena (real time operating system) su namenjeni za primene računara u kojima je neophodno obezbediti reakciju na vanjski događaj u unapred zadanom vremenu. Ovakvi operativni sistemi su, zbog toga, podređeni ostvarenju što veće brzine izvršavanja korisničkih programa. Za operativne sisteme realnog vremena je tipično da su, zajedno sa računarom, ugrađeni (embedded) u sistem, čije ponašanje se ili samo prati, ili čijim ponašanjem se upravlja. Zadatak operativnih sistema realnog vremena je da samo stvore okruženje za korisničke programe, jer komunikaciju sa krajnjim korisnikom obavljaju korisnički programi. Zato se operativni sistemi realnog vremena obično koriste samo na programskom nivou.

174. Šta karakteriše multiprocesorske operativne sisteme?

Multiprocesorski operativni sistemi upravljaju računarskim sistemom sa više procesora opšte namene, koji pristupaju zajedničkoj radnoj memoriji. Podrazumeva se da ove procesore i radnu memoriju povezuje sabirnica.

Specifičnosti multiprocesorskog operativnog sistema su vezane za modul za rukovanje procesorom i posledica su istovremene aktivnosti više procesa na raznim procesorima. Zato se sinhronizacija procesa u ovakvim okolnostima više ne može zasnivati na onemogućenju prekida, nego na zauzimanju sabirnice, jer je to jedini način da se spreči da više od jednog procesa pristupa istoj lokaciji radne memorije. Mogućnost istovremene aktivnosti više procesa na raznim procesorima usložnjava raspoređivanje,

jer ono mora da odabere ne samo proces, koji će biti aktivan, nego i da odabere procesor, koji će da se preključi na odabrani proces.

175. Koje module sadrži mikrokernel?

Mikrokernel sadrži:

- modul za rukovanje procesima,
- modul za razmjenu poruka,
- modul za rukovanje radnom memorijom,
- modul za rukovanje kontrolerima,
- modul za rukovanje procesorom.

Mikrokernel ne sadrži modul za rukovanje datotekama

176. Šta karakteriše poziv udaljene operacije (RPC)?

Ako pozivana operacija ne odgovara potprogramu koji se lokalno izvršava u okviru aktivnosti procesa pozivaoca, nego odgovara potprogramu koji se izvršava u okviru aktivnosti drugog, udaljenog procesa, aktivnog na udaljenom računaru, riječ je o pozivu udaljene operacije (Remote Procedure Call - RPC). Proces koji poziva udaljenu operaciju se nalazi u ulozi klijenta (primaoca usluge), a proces koji obavlja udaljenu operaciju se nalazi u ulozi servera (davaoca usluge). Poziv udaljene operacije ima oblik poziva potprograma, u kome se navode oznaka (ime) operacije i njeni argumenti. Ovakav potprogram se naziva klijentski potprogram, jer je klijent njegov jedini pozivalac.

177. Šta radi klijentski potprogram?

U klijentskom potprogramu je sakriven niz koraka, koji se obavljaju, radi dobijanja zahtevane usluge. U ove korake spadaju: 1. pronalaženje procesa servera, koji pruža zahtevanu uslugu, 2. pakovanje (marshalling) argumenata (navedenih u pozivu klijentskog potprograma) u poruku zahteva, 3. slanje serveru ove poruke zahteva, 4. prijem od servera poruke odgovora sa rezultatom pružanja zahtevane usluge, 5. raspakivanje prispele poruke odgovora i 6. isporuka rezultata pružanja zahtevane usluge pozivaocu klijentskog potprograma.

178. Za šta su zaduženi serverski potprogrami?

Simetrično klijentskom potprogramu postoje dva serverska potprograma (server stub). Njih poziva jedino server, a oni kriju više koraka, koji se obavljaju, radi pružanja zahtevane usluge. Prvi od serverskih potprograma obuhvata:

1. prijem poruke zahteva i
2. raspakivanje argumenata iz ove poruke.

Drugi od serverskih potprograma obuhvata:

1. pakovanje rezultata usluge (koju je pružio server) u poruku odgovora i
2. slanje klijentu ove poruke odgovora.

Između poziva ova dva serverska potprograma se nalazi lokalni poziv operacije, koja odgovara zahtevanoj usluzi, odnosno, programski tekst, koji opisuje aktivnost servera na pružanju zahtevane usluge

179. Koji problemi su vezani za poziv udaljene operacije?

Uprkos nastojanju da što više liči na poziv lokalne operacije, poziv udaljene operacije se značajno razlikuje od svog uzora. Te razlike su posledica koraka, sakrivenih u pozivu udaljene operacije, koji uzrokuju da se u toku poziva udaljene operacije mogu da pojave problemi, čija pojava nije moguća kod poziva lokalne operacije. Tako je moguće:

1. da se ne pronađe server, koji pruža zahtevanu uslugu,
2. da se, u toku prenosa, izgube ili poruka zahteva ili poruka odgovora, kao i
3. da dođe do otkaza ili servera, ili klijenta u toku njihovog rada.

180. Šta podrazumeva dinamičko linkovanje klijenta i servera?

Činjenica da se u okviru klijentskog potprograma javlja potreba za pronalaženjem servera, ukazuje da u vrijeme pravljenja izvršnog oblika klijentskog programa nije poznato koji server će usluživati klijenta. U opštem slučaju, može biti više servera iste vrste i svaki od njih može istom klijentu da pruži zatraženu uslugu. Radi toga se uvodi poseban server imena (name server, binder). Njemu se, na početku svoje aktivnosti, obraćaju svi serveri i ostavljaju podatke o sebi, kao što je, na primjer, podatak o vrsti usluge koje pružaju. Serveru imena se obraćaju i klijenti, radi pronalaženja servera, koji pruža zahtijevanu uslugu. Na ovaj način se ostvaruje dinamičko linkovanje (dynamic binding) klijenta, koji zahtijeva uslugu, i servera, koji pruža zahtijevanu uslugu.

181. Koje operacije podržava protokol razmene poruka između klijenta i servera?

Protokol razmjene poruka između klijenta i servera podrazumijeva:

- sistemsku operaciju zahtijevanja usluge
- sistemsku operaciju prijema zahteva
- sistemsku operaciju slanja odgovora

182. Za šta su zadužene sistemske operacije koje ostvaruju protokol razmene poruka?

Sistemska operacija zahtijevanja usluge zaustavlja aktivnost klijenta do stizanja odgovora ili do isticanja zadanog vremenskog perioda. Sistemska operacija prijema

zahtjeva zaustavlja aktivnost servera do stizanja zahtjeva, a sistemska operacija slanja odgovora zaustavlja aktivnost servera do isporuke odgovora ili do isticanja zadanog vremenskog intervala. Ove tri sistemske operacije su zadužene za prenos poruka. Pored slanja i prijema poruka, one: 1. potvrđuju prijem poruka 2. retransmituju poruke, čiji prijem nije potvrđen 3. šalju upravljačke poruke, kojim se provjerava i potvrđuje aktivnost servera. U nadležnosti ovih operacija je i rastavljanje poruka u pakete, sastavljanje poruka od paketa, potvrda prijema paketa i retransmisija paketa čiji prijem nije potvrđen, kao i prilagođenje brzine slanja paketa brzini kojom oni mogu biti primani. Koriste usluge drajvera sata. One pozivaju i (neblokirajuće) operacije gornjeg dijela drajvera mrežnog kontrolera. Takođe, brinu se i o baferima, najmjenjenim za (privremeno) smeštanje poruka.

183. Šta sadrže poruke koje razmenjuju klijent i server?

Svaka od poruka, koje se razmenjuju između procesa, se sastoji:

1. od upravljačkog dela poruke i
2. od sadržaja poruke.

Upravljački deo poruke obuhvata:

1. adresu odredišnog procesa (kome se poruka upućuje),
2. adresu izvorišnog procesa (od koga poruka kreće, a kome se, eventualno, kasnije upućuje odgovor) i
3. opis poruke (njenu vrstu, njen redni broj i slično).

184. Šta je potrebno za sigurnu razmenu poruka između klijenta i servera?

Komunikacione linije su pristupačne svim korisnicima, pa je svaki od njih u poziciji da preuzima tuđe poruke i da šalje poruke u tuđe ime. Sprječavanje preuzimanja tuđih poruka zasniva se na kriptovanju poruka, a sprječavanje slanja poruka u tuđe ime se zasniva na nedvosmislenoj međusobnoj identifikaciji procesa. Bitnu ulogu u sigurnosti razmjene poruka igra i poseban server, koji se naziva povjerenik. On posjeduje unaprijed dogovoren poseban ključ kriptovanja za komunikaciju sa svakim procesom. Zahvaljujući tome, klijent može da pošalje poruku povjereniku, koja sadrži ime klijenta i ime servera sa kojim klijent želi da ostvari sigurnu komunikaciju. Povjerenik šalje samo klijentu i serveru interni ključ kriptovanja za sigurnu komunikaciju i ujedno se obavi njihova međusobna identifikacija, tako da se drugi procesi ne mogu umiješati u njihovu komunikaciju. Kod asimetrične kriptografije, uloga povjerenika je da čuva javne ključeve i tako osigura međusobnu identifikaciju procesa.

185. Šta karakteriše digitalni potpis?

. Digitalni potpis (digital signature) se šalje uz poruku. On sadrži podatke koji jednoznačno reprezentuju poruku, pa predstavljaju otisak prsta poruke (fingerprint, cryptographic checksum). Otisak prsta poruke formiraju jednosmerne funkcije (one-way functions) na osnovu sadržaja poruke. Digitalni potpis nastane kada se otisak prsta poruke dekriptuje (transformiše) primenom algoritma dekriptovanja i privatnog ključa. Primalac poruke kriptuje (retransformiše) digitalni potpis primenom algoritma kriptovanja i javnog ključa. Ako se rezultat kriptovanja digitalnog potpisa poklapa sa otiskom prsta primljene poruke, tada je poruka nedvosmisleno stigla od pošiljaoca.

186. Od čega zavisi propusnost servera?

??? Propusnost servera zavisi od sekvencijalnosti njegove aktivnosti, koja znatno manje zavisi od propusnosti i brzine pružanja usluga.

187. Šta sadrže dozvole na kojima se zasniva zaštita datoteka u distribuiranom sistemu?

Da bi klijent dobio neku uslugu, on mora da poseduje odgovarajuću dozvolu, koju prosleđuje serveru u okviru zahteva za uslugom. Dozvola sadrži: 1. redni broj servera, 2. redni broj deskriptora datoteke (odnosno, deskriptora imenika), 3. oznaku vrste usluge i 4. oznaku ispravnosti dozvole.

188. Šta karakteriše distribuiranu sinhronizaciju?

Saradnja procesa, aktivnih na raznim računarima, zahtijeva njihovu sinhronizaciju, što se ostvaruje razmjenom poruka. Najjednostavniji način za ostvarenje sinhronizacije se zasniva na uvođenju procesa koordinatora. Njemu se obraćaju svi procesi, zainteresovani za sinhronizaciju, a koordinator donosi odluke o njihovoj sinhronizaciji. Proces koordinator spada u centralizovane algoritme sinhronizacije. Postoje i distribuirani algoritmi sinhronizacije koji se zasnivaju na međusobnom dogovaranju procesa, zainteresovanih za sinhronizaciju. Ovaj algoritam ima veću razmjenu poruka i komplikovaniji je od centralizovanih algoritama, ali nema veći praktični značaj. Za distribuirane operativne sisteme nije samo bitno da omoguće efikasnu sinhronizaciju procesa, nego i da podrže poseban oblik sinhronizacije procesa, koji obezbjeđuje da se obave ili sve operacije iz nekog niza pojedinačnih akcija ili nijedna od njih. Ovakav niz operacija se naziva transakcija, a transakcije koje imaju svojstvo da se obave u cjelosti ili nikako, nazivaju se atomske transakcije. Distribuirani računarski sistemi su neefikasniji kod pojave mrtve petlje, nego centralizovani (jednoprocesorski) računar.

189. Šta karakteriše distribuirani računarski sistem?

Distribuirani računarski sistem je zamišljen tako da integriše mnoštvo računara u moćan multiračunarski sistem. Na taj način je moguće od više jeftinih i malih računara napraviti moćan multiračunarski sistem povoljne cijene. Ovakav multiračunarski sistem, uz to, nudi i veću pouzdanost, jer kvar pojedinačnog računara nije fatalan za cio sistem, kao i mogućnost proširenja, jer je moguće naknadno dodavanje računara u

sistem. Pored integrisanja pojedinačnih računara u multiračunarski sistem, distribuirani računarski sistem omogućuje i dijeljenje skupih resursa ovakvog multiračunarskog sistema između više korisnika, a nudi i prilagodljivost zahtevima korisnika, željenu raspoloživost i predvidivost odziva, pa, čak, i veću sigurnost, jer korisnici mogu da čuvaju povjerljive podatke na svom računaru, koga fizički štite i čije korišćenje mogu da kontrolišu.

190. Šta karakteriše distribuiranu softversku platformu?

Distribuirana softverska platforma je obično specijalizovana tako da nudi konzistentan skup operacija, koje omogućuju razvoj željene vrste distribuiranih softverskih sistema. Ovakav skup operacija podržava saradnju komponenti koje obrazuju distribuirani softverski sistem. U tom pogledu se može govoriti o raznim distribuiranim softverskim platformama, poput one namenjene za podršku distribuiranih dokumenata (World Wide Web), ili one namenjene za podršku distribuiranom objektno orijentisanom programiranju (CORBA, Java middleware, .NET). U osnovi ovakvih sistema se krije klijent-server model. Server se nalazi na strani rukovaoca distribuiranim dokumentima (web site) ili rukovaoca objektima, a klijent se nalazi na strani korisnika distribuiranih dokumenata (web browser) ili pozivaoca operacija udaljenih objekata.

191. Od čega se sastoje komande znakovnog komandnog jezika?

Komanda znakovnog jezika započinje operatorom (npr. u obliku putanje izvršne datoteke, koja opisuje rukovanje), a završava operandom ili operandima (npr. u obliku putanja datoteka, kojima se rukuje). Primjer / objašnjenje: kopiraj godina1.txt godina2.txt predstavlja primjer prethodno opisane komande znakovnog komandnog jezika. U ovom primeru se pretpostavlja da radni imenik obuhvata izvršnu datoteku sa imenom kopiraj.bin i tekst datoteku godina1.txt. Takođe se pretpostavlja da izvršavanje programa iz izvršne datoteke kopiraj.bin dovodi do stvaranja datoteke godina2.txt, koja je po sadržaju identična datoteci godina1.txt i koja pripada radnom imeniku. Prethodna komanda opisuje korisnu operaciju, ako svi studenti prve godine studija upisuju drugu godinu studija.

192. Kako se zadaju komande grafičkih komandnih jezika?

Komandni jezik može olakšati zadavanje komandi, ako omogućí korisniku da operator komande bira u spisku operatora (menu), umesto da ga pamti i u celosti navodi. Spisak operatora se prikazuje na ekranu, a izbor operatora se vrši pomoću namenskih dirki tastature ili miša. Nakon izbora operatora sledi, po potrebi, dijalog u kome korisnik navodi (ili opet bira) operand (operande) komande. Ovakvi komandni jezici se nazivaju grafički komandni jezici (menu driven user interface, graphical user interface). Oni još više pojednostavljaju zadavanje komandi, ako korisniku omogućuju da ne bira operator, nego samo operande komandi.

193. Šta su ciljevi znakovnih komandnih jezika?

Ciljevi znakovnih komandnih jezika obuhvataju: 1. omogućavanje izvršavanja pojedinih (korisničkih) programa, 2. omogućavanje kombinovanja izvršavanja više (korisničkih) programa i 3. omogućavanje pravljenja komandnih datoteka (command file, shell script).

194. Šta omogućuju znakovni komandni jezici?

Znakovni komandni jezici omogućuju izvršavanje pojedinih programa, kombinovanje izvršavanja više korisničkih programa i pravljenje komandnih datoteka.

195. Šta omogućuje preusmeravanje?

Zahvaljujući obradi znakova komande, moguće je interpreteru znakovnog komandnog jezika saopštiti i da preusmeri (redirect) standardni ulaz i standardni izlaz sa tastature i ekrana na proizvoljno odabrane datoteke. Ovo je važno za pozadinske procese, koji nisu u interakciji sa korisnikom. Zahvaljujući preusmeravanju, pozadinski proces ne ometa interaktivni rad korisnika, jer, umesto tastature i ekrana, koristi odabrane datoteke. Uobičajeno je da preusmeravanje standardnog ulaza najavljuje znak <, a da preusmeravanje standardnog izlaza (kao i standardnog izlaza greške) najavljuje znak >.

Primjer / objašnjenje

komanda:

kompiriraj < program.c > program.objx

saopštava interpreteru znakovnog komandnog jezika da stvori proces na osnovu izvršne datoteke kompiriraj.bin, pri čemu kao standardni ulaz procesa služi datoteka program.c, a kao njegov standardni izlaz datoteka program.obj. U ovom primeru ekran i dalje služi kao standardni izlaz greške. Izvršavanje prethodne komande omogućuje kompilaciju C programa, sadržanog u datoteci program.c. Rezultat kompilacije se smešta u datoteku program.obj, a eventualne poruke o greškama kompilacije se prikazuju na ekranu.

196. Čemu služi pipe?

Umesto preusmeravanja standardnog ulaza i standardnog izlaza, moguće je nadovezati standardni izlaz jednog procesa na standardni ulaz drugog procesa i tako obrazovati tok procesa (pipe). Nadovezivanje u tok se označava pomoću znaka |.

197. Čemu služi baferovana specijalna datoteka?

Razmena podataka između dva procesa, koji su povezani u tok, se ostvaruje posredstvom posebne specijalne datoteke. Njoj odgovara bafer u radnoj memoriji. Ova baferovana specijalna datoteka služi prvom od ovih procesa kao standardni izlaz, a drugom od njih kao standardni ulaz. Znači, prvi proces samo piše u ovu datoteku, a

drugi samo čita iz nje. Prilikom obrazovanja toka procesa, interpreter znakovnog komandnog jezika stvara procese, koji se povezuju u tok. Pri tome on koristi istu posebnu specijalnu datoteku kao standardni izlaz i ulaz za svaki od parova ovih procesa. Zatim interpreter znakovnog komandnog jezika čeka na kraj aktivnosti poslednjeg od ovih procesa.

198. Šta karakteriše pozadinske procese?

Pozadinski procesi se razlikuju od običnih (interaktivnih) procesa po tome što interpreter znakovnog komandnog jezika, nakon stvaranja pozadinskog procesa, ne čeka kraj njegove aktivnosti, nego nastavlja interakciju sa korisnikom. Zato su pozadinski procesi u principu neinteraktivni.

199. Šta karakteriše komandne datoteke?

One opisuju okolnosti pod kojima se izvršavaju korisnički programi, a sadržaj komandne datoteke preuzima na interpretiranje interpreter znakovnog komandnog jezika. Komandne datoteke imaju poseban tip, da bi ih interpreter znakovnog komandnog jezika mogao prepoznati. Zahvaljujući tome, ime svake komandne datoteke, uostalom, kao i ime svake izvršne datoteke, predstavlja ispravnu komandu znakovnog komandnog jezika. Iako broj i vrste ovakvih komandi nisu ograničeni, jer zavise samo od kreativnosti i potreba korisnika, ipak je moguće napraviti njihovu klasifikaciju i navesti neke neizbježne grupe komandi. Najgrublja podjela je na korisničke komande i administratorske komande.

200. Šta omogućuju korisničke komande?

Korisničke komande, između ostalog, omogućuju:

1. rukovanje datotekama,
2. rukovanje imenicima,
3. rukovanje procesima i
4. razmenu poruka između korisnika.

201. Šta omogućuju administratorske komande?

Administratorske komande omogućuju:

1. pokretanje i zaustavljanje rada računara,
2. spašavanje (backup) i vraćanje (restore) datoteka,
3. rukovanje vremenom,
4. sabijanje (compaction) datoteka,
5. ažuriranje podataka o korisnicima računara i njihovim pravima,

6. generisanje izveštaja o korišćenju računara (o korišćenju procesorskog vremena ili o korišćenju prostora na disku),

7. rukovanje konfiguracijom računara (određivanje načina rada uređaja i programa koji ulaze u njegov sastav),

8. proveru ispravnosti rada računara i

9. pripremu diskova za korišćenje (ovo obuhvata pronalaženje oštećenih blokova i njihovo isključivanje iz upotrebe, pronalaženje izgubljenih blokova i njihovo uključivanje u evidenciju slobodnih blokova, formiranje skupa datoteka na disku i njegovo uključivanje u skup datoteka računara).

202. Čime se bavi sigurnost?

Sigurnost se bavi načinima prepoznavanja ili identifikacije korisnika (authentication), kao i načinima provere njihovih prava pristupa (authorization). Operativni sistem nudi mehanizme sigurnosti pomoću kojih mogu da se ostvare različite politike sigurnosti.

203. Šta omogućuju sistemske operacije za rukovanje procesima?

- Stvaranje procesa
- Uništenje procesa
- Izmjenu atributa procesa(npr. izmjena radnog imenika procesa)

204. Šta obuhvata stvaranje procesa?

Stvaranje procesa obuhvata stvaranje njegove slike i njegovog deskriptora, kao i pokretanje njegove aktivnosti.

205. Šta obuhvata uništenje procesa?

Uništenje procesa obuhvata zaustavljanje njegove aktivnosti, kao i uništenje njegove slike i njegovog deskriptora.

206. Šta sadrži slika procesa?

- Izvršavane mašinske naredbe
- Promjenljive
- Stek

Ona obuhvata niz lokacija radne memorije sa uzastopnim (logičkim) adresama.

207. Za šta se koristi slobodna radna memorije procesa?

- Za širenje(punjenje) steka
- Za stvaranje dinamičkih promjenljivih(heap)

208. Koji atributi procesa postoje?

. Ovi atributi karakterišu aktivnost procesa. Oni obuhvataju:

1. stanje procesa ("spreman", "aktivan", "čeka"),
2. sadržaje procesorskih registara (zatečene u njima pre poslednjeg preključivanja procesora sa procesa),
3. numeričku oznaku vlasnika procesa,
4. oznaku procesa stvaraoca,
5. trenutak pokretanja aktivnosti procesa,
6. ukupno trajanje aktivnosti procesa (odnosno, ukupno vreme angažovanja procesora),
7. podatke o slici procesa (njenoj veličini i njenom položaju u radnoj i masovnoj memoriji),
8. podatke o datotekama koje proces koristi,
9. podatak o radnom imeniku procesa i
10. razne podatke neophodne za upravljanje aktivnošću procesa (poput prioriteta procesa ili položaja sistemskog steka procesa, koga koristi operativni sistem u toku obavljanja sistemskih operacija).

209. Koje sistemske operacije za rukovanje procesima postoje?

- Operacije stvaranja procesa (fork() i exec())
- Operacije uništenja procesa (exit())
- Posebna operacija wait() — omogućuje zaustavljanje aktivnosti procesa stvaraoca i preključivanje procesora na stvarani proces

210. Koji se atributi nasleđuju od procesa stvaraoca prilikom stvaranja procesa?

- Numerička oznaka vlasnika procesa
- Podatak o radnom imeniku procesa
- Njegov prioritet

211. Koji se atributi procesa nastanu prilikom njegovog stvaranja?

- Npr. podaci o slici procesa

212. Šta karakteriše kopiju slike procesa?

Kopija slike procesa se nalazi u masovnoj memoriji. U toku aktivnosti procesa, mijenja se samo dio njegove slike u radnoj memoriji, jer se mijenjaju samo vrijednosti njegovih promjenljivih i njegov stek, pa je prilikom izbacivanja slike procesa, potrebno samo njen izmijenjeni dio prebaciti u kopiju slike u masovnoj memoriji. Ali, pri vraćanju slike u radnu memoriju prebacuje se cijela njena kopija, da bi se u radnoj memoriji obnovila cijela slika procesa. Vraćanje slike jednog procesa vezano je za uništavanje drugog procesa, jer se tada oslobađa prostor u radnoj memoriji. Podaci o kopiji slike procesa (koja nastaje istovremeno sa slikom procesa ili prilikom njenog prvog izbacivanja, a nestaje istovremeno sa slikom procesa) čuvaju se u deskriptoru procesa, zajedno sa podacima o slici procesa.

213. Koje raspoređivanje je vezano za zamenu slika/stranica procesa?

Za zamjenu slike procesa vezano je dugoročno raspoređivanje, u okviru koga se odabira proces čija se slika izbacuje, kao i proces čija se slika ubacuje.

214. Šta karakteriše rukovanje nitima unutar operativnog sistema?

Rukovanje nitima može, ali i ne mora, biti u nadležnosti sloja za rukovanje procesima. Kada je rukovanje nitima povjereno sloju za rukovanje procesima, tada operativni sistem nudi sistemske operacije za rukovanje nitima, koje omogućuju stvaranje, uništavanje i sinhronizaciju niti. U ovom slučaju, deskriptori i sistemski stek niti se nalaze u sistemskom prostoru, dok se sopstveni stek niti nalazi u korisničkom prostoru (unutar slike procesa).

215. Šta karakteriše rukovanje nitima van operativnog sistema?

U slučaju kada rukovanje nitima nije u nadležnosti operativnog sistema, brigu o nitima potpuno preuzima konkurentna biblioteka. Pošto ona pripada slici procesa, rukovanje nitima u ovom slučaju se potpuno odvija u korisničkom prostoru, u kome se nalaze i deskriptori niti, kao i stekovi niti. Osnovna prednost rukovanja nitima van operativnog sistema (ovo se još zovu i “zelene niti”) je efikasnost, jer su pozivi potprograma konkurentne biblioteke brži (kraći) od poziva sistemskih operacija. Mana je što poziv blokirajuće sistemske operacije iz jedne niti dovodi do zaustavljanja aktivnosti procesa kome ta nit pripada. Na taj način se sprječava konkurentnost unutar procesa, jer zaustavljanje aktivnosti procesa sprečava aktivnost njegovih spremnih niti. Ovo značajno umanjuje praktičnu vrijednost ovog pristupa.

216. Šta karakteriše nulti proces?

Tipičan primer sistemskog procesa je nulti ili beskonačni (idle) proces, na koga se procesor preključuje, kada ne postoji drugi spreman proces. Znači, beskonačan proces ima zadatak da zaposli procesor, kada nema mogućnosti za korisnu upotrebu procesora. U toku aktivnosti beskonačnog procesa izvršava se beskonačna petlja, što znači da je beskonačan proces uvek ili spreman ili aktivan (on ne prelazi u stanje

čekanja). Njegov prioritet je niži od prioriteta svih ostalih procesa, a on postoji za sve vreme aktivnosti operativnog sistema.

217. Šta je karakteristično za proces dugoročni raspoređivač?

Još jedan primjer sistemskog procesa je proces dugoročni raspoređivač (swapper), koji se brine o zamjeni slika procesa (kada za to ima potrebe). On se periodično aktivira, radi pozivanja operacije za zamjenu slika procesa. Nakon toga, ovaj proces se uspava, odnosno, njegova aktivnost se zaustavlja, dok ne nastupi trenutak za njegovo novo aktiviranje. Proces dugoročni raspoređivač postoji za sve vrijeme aktivnosti operativnog sistema (ako ima potrebe za dugoročnim raspoređivanjem).

218. Šta radi proces identifikator?

U sistemске procese spada i proces identifikator (login process), koji podržava predstavljanje korisnika. Proces identifikator opslužuje terminal, da bi posredstvom njega stupio u interakciju sa korisnikom u toku predstavljanja, radi preuzimanja imena (koje se prikazuje na ekranu) i lozinke (koja se ne prikazuje na ekranu) korisnika.

219. Ko stvara proces komunikator?

Po preuzimanju imena i lozinke, proces identifikator provjerava njihovu ispravnost i, ako je prepoznao korisnika, tada stvara proces komunikator, koji nastavlja interakciju sa korisnikom.

220. Šta označava SUID (switch user identification)?

Pošto korisnici nemaju načina da pristupe datoteci lozinki, postavlja se pitanje kako omogućiti korisniku da sam izmijeni svoju lozinku. Ako je administrator vlasnik izvršne datoteke sa inicijalnom slikom procesa za izmenu lozinki, dovoljno je naznačiti da on treba da bude i vlasnik procesa nastalog na osnovu ove izvršne datoteke (SUID). Zahvaljujući tome, vlasnik ovakvog procesa je administrator bez obzira koji je korisnik pokrenuo proces. U tom slučaju, nema smetnje da korisnik, koji izazove stvaranje procesa za izmjenu lozinke, pristupi datoteci lozinki. Pri tome, proces za izmjenu lozinki dozvoljava korisniku samo da izmeni sopstvenu lozinku, tako što od korisnika dobije ime, važeću i novu lozinku, pa ako su ime i važeća lozinka ispravni, on važeću lozinku zamijeni novom.

221. Šta karakteriše heš (hash/jednosmernu) funkciju?

???? Prepisano iz skripte, nema u knjizi

Kripto-sistemi se grade od kripto-primitiva, bazičnih matematičkih konstruktora koji nam omogućavaju da operišemo. Jedan od kripto-primitiva je heš funkcija. Postoje kriptografske i opšte heš funkcije. Heš funkciju karakteriše nekoliko osobina:

1. Jednostruko preslikavanje — Heš funkcija je dizajnirana da efikasno mapira ulaznu poruku proizvoljne dužine na fiksnu dužinu, poznatu kao heš vrijednost, ili heš kod. Ova operacija je jednosmijerna, što znači da je teško ili nemoguće povratiti originalnu poruku iz heš vrednosti. Heš funkcija je dizajnirana tako da mala promjena u ulaznoj poruci proizvodi potpuno drugačiju heš vrijednost.
2. Jedinstvenost — heš funkcija treba da generiše jedinstvenu heš vrijednost za svaku različitu ulaznu poruku, ali to često nije slučaj i tada se javljaju "heš kolizije". Dobre heš funkcije imaju veoma malu količinu kolizija. (Ne postoji heš funkcija bez kolizija jer ti heš funkcijom mapiraš prostor većih dimenzija (fajl od 52 GB) na manji fiksni prostor (512 bitova npr.))
3. Brzina izračunavanja — heš f-je su dizajnirane da budu brze i efikasne
4. Determinizam — isti ulaz uvijek generiše istu heš vrijednost
5. Osjetljivost na promjene — mala promjena ulaza može dovesti do velike promjene u heš vrijednosti. Kvalitetne heš funkcije trebaju ispunjavati ove karakteristike kako bi pružile adekvatnu sigurnost i pouzdanost u širokom spektru primjena, uključujući digitalne potpise, provjeru autentičnosti, provjeru lozinki, sigurno skladištenje podataka itd.

222. Šta karakteriše simetričnu kriptografiju?

Ako algoritam dekriptovanja direktno i jednoznačno sledi iz algoritma kriptovanja, a ključ dekriptovanja iz ključa kriptovanja, riječ je o simetričnoj kriptografiji. Poznavanje ključa kriptovanja omogućuje i dekriptovanje, tako da ključ kriptovanja predstavlja tajnu kao i ključ dekriptovanja. Algoritmi kriptovanja i dekriptovanja ne predstavljaju tajnu, jer je praksa pokazala da se takva tajna ne može sačuvati. Simetrična kriptografija nije podesna za kriptovanje poruka. U praksi, simetrična kriptografija se zasniva na DES i IDEA pristupima .

223. Šta karakteriše asimetričnu kriptografiju?

U asimetričnoj kriptografiji, iz ključa kriptovanja ne može se odrediti ključ dekriptovanja, pa poznavanje ključa kriptovanja ne omogućuje dekriptovanje. Ključ kriptovanja se naziva javni ključ, jer je dostupan svima, a ključ dekriptovanja privatni ključ, jer je dostupan samo osobama ovlašćenim za dekriptovanje. Algoritmi kriptovanja su jednostavni, a njima odgovaraju komplikovani algoritmi dekriptovanja, pa je asimetrična kriptografija mnogo sporija od simetrične. U praksi, asimetrična kriptografija se zasniva na RSA algoritmu.

224. Na čemu se temelji tajnost kriptovanja?

Tajnost kriptovanja teksta se zasniva na činjenici da (1) komplikovanost algoritama kriptovanja i dekriptovanja, (2) velika dužina ključeva kriptovanja i dekriptovanja, kao i

(3) veliki broj ovih ključeva čine praktično neizvodljivim pokušaj da se "na silu" dekriptuje kriptovani tekst probanjem, jednog po jednog, svih mogućih ključeva dekriptovanja.